

Git & GitHub Notes

Introduction to Git

Git is a distributed version control system used to track changes in source code.

Why Use Git?

- Tracks code changes
- Supports teamwork
- Maintains project history
- Easy backup and recovery

Developed By **Linus Torvalds**

Important Terms

Repository → Project storage

Commit → Saved changes

Branch → Separate development line

Clone → Copy repository

Exam Tip: Git is used locally, GitHub is cloud based.

Basic Git Workflow



Edit Code → **git add** → **git commit** → Push to GitHub

@NeerajSingh-DevOps

Git & GitHub Notes

Basic Git Commands

Initialize Repository

`git init`

Creates a new Git repository.

Check Status

`git status`

Shows modified and tracked files.

Add Files

`git add .`

`git add filename`

Commit Changes

`git commit -m "message"`

View Commit History

`git log`

Clone Repository

`git clone URL`

Copies remote repository to local system.

Command	Purpose
<code>git init</code>	Create repository
<code>git add</code>	Stage files
<code>git commit</code>	Save changes
<code>git log</code>	Show history

Exam Tip:
Always commit
with meaningful
messages.

Create File → `git add` → `git commit` → Repository Updated

Git & GitHub Notes

Branches and GitHub

A branch is a separate line of development used to work independently.

Create Branch

git **branch** feature

Switch Branch

git **checkout** feature

Merge Branch

git **merge** feature

Merges changes from one branch into another.

Connect with GitHub

git **remote add** origin URL

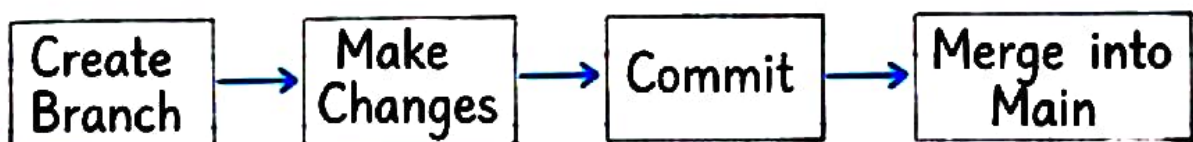
Push Code to GitHub

git **push origin main**

Git → Local version control

GitHub → Online hosting platform

Exam Tip:
Branches help developers work safely without affecting main code.



Git & GitHub Notes

Pull, Fetch and Merge

Git allows developers to synchronize local and remote repositories.

git pull

```
git pull origin main
```

Downloads and merges latest changes from GitHub.

git fetch

```
git fetch
```

Downloads updates without merging.

git merge

```
git merge branch_name
```

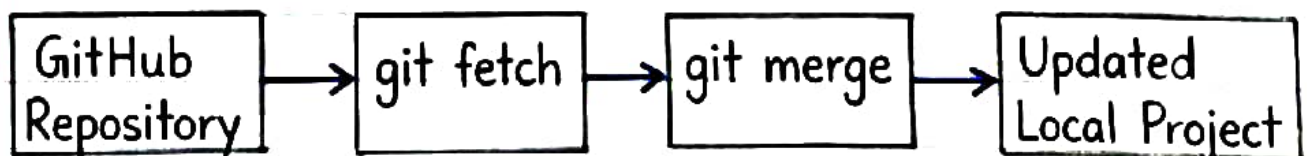
Combines changes from another branch.

Command	Purpose
git pull	Fetch + Merge
git fetch	Download changes
git merge	Combine branches

Merge Conflict

Happens when two developers edit same lines of code.

Exam Tip: Always pull latest changes before pushing code



Git & GitHub Notes

GitHub Workflow and Best Practices

Basic GitHub Workflow

1. Create Repository →
2. Clone Repository →
3. Make Changes →
4. `git add .` ←
5. `git commit -m "message"` ←
6. `git push origin main` ←

Important Git Commands

`git status` → Shows current state of repo

`git log` → Displays commit history

`git diff` → Shows changes between versions

`git reset` → Undoes changes or resets HEAD

Best Practices

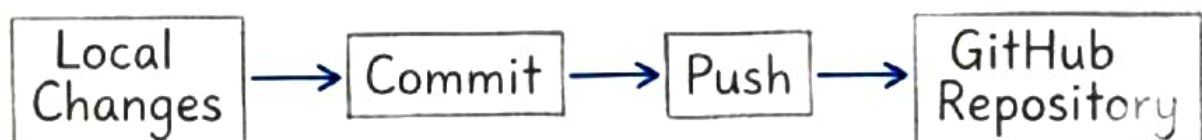
- Write meaningful commit messages
- Pull latest changes before push
- Use branches for new features
- Keep repository organized

.gitignore File

Used to ignore unnecessary files from repository.

node_modules/
.env

Exam Tip: Never upload passwords or secret keys to GitHub.



Git & GitHub Notes

Git Cheat Sheet

Repository Commands

- git init - Used to correct repository.
- git clone URL - Cone to swm from oreraing URL.
- git status - Ship for status of mow repository.

Staging and Commit

- git add .
- git commit -m "message"

Branch Commands

- git branch
- git checkout branch_name
- git merge branch_name

Remote Repository Commands

- git remote add origin URL
- git push origin main
- git pull origin main
- Used to connect and sync with GitHub.

History Commands

- git log
- git diff

Command	Function
git init	Initialize repo
git add	Stage files
git commit	Save changes
git push	Upload code
git pull	Download updates

Exam Tip:

Practice commands daily to remember workflow.

Edit Code → Add → Commit → Push → GitHub

Git & GitHub Notes

GitHub Collaboration

GitHub collaboration allows multiple developers to work on the same project.

Fork Repository

Creates a personal copy of another repository. **Fork**

Pull Request (PR)

A **request** to **merge** changes into the main project.

Collaborator Workflow

1. Fork Repository → 1
2. Clone Repository → 2
3. Create Branch → 3
4. Make Changes → 4
5. Commit Changes → 5
6. Push Code → 6
7. Create Pull Request → 7

Code Review

Team members review code before merging.

Issues

Used to track bugs, improvements, and tasks.

Feature	Purpose
Fork	Copy repository
Pull Request	Request merge
Issues	Track tasks

Exam Tip: Pull Requests improve teamwork and code quality.

Developer → Fork → Edit Code → Pull Request → Merge

Git & GitHub Notes

Git Interview Questions

Q1. What is Git?

Git is a distributed version control system used to track code changes.

Q2. Difference between Git and GitHub?

Git → Local version control

GitHub → Cloud hosting platform

Q3. What is a **repository**?

A **repository** stores project files and history.

Q4. What is **commit** in Git?

A **commit** saves changes with a unique ID.

Q5. What is branching?

Branching allows independent development without affecting main code.

Q6. What is **merge conflict**?

Happens when multiple changes affect same code lines.

Q7. Difference between git pull and git fetch?

git pull → fetch + merge

git fetch → only downloads changes

Interview Tip:
Explain workflow
with real examples.

Code Changes → Commit → Push → GitHub Repository

Git & GitHub Notes

Git Errors and Troubleshooting

Common Git Errors

1. merge conflict
2. detached HEAD
3. repository not found
4. permission denied
5. failed to push some refs

Merge Conflict

Occurs when Git cannot automatically combine changes.

- Open conflicting files
- Edit manually
- Commit again

Detached HEAD

HEAD points to a commit instead of branch.

`git checkout main`

Authentication Error

Wrong password or token while pushing code.

Useful Troubleshooting Commands

`git status`

`git log`

`git diff`

`git reset --hard`

Exam Tip: `git status` is the most useful debugging command.

Error → Check Status → Fix Problem → Commit Again

Git & GitHub Notes

GitHub Security and Best Practices

GitHub Security Tips

- Never upload passwords
- Keep repositories organized
- Use strong passwords
- Enable Two-Factor Authentication (2FA)
- Use private repositories for sensitive projects

.gitignore File

Used to prevent unnecessary or sensitive files from being uploaded.

```
.gitignore
.env
node_modules/
*.log
```

Best Practices

- Write clean commit messages
- Create separate branches
- Pull latest changes regularly
- Backup important repositories

Exam Tip:
Never share API
keys publicly
on GitHub.

GitHub README

A README file explains project details and usage.

- Project Name
- Features
- Installation
- Usage

